МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 5

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23 Враженко Д. О.

_____
(подпись студента)

Принял преподаватель Тесла-Блозва С. А.

_____
(подпись руководителя)

Работа представлена «____»_____ 2026 г.

Допущен к работе «___» _____ 2026 г.

Москва 2026

-

Оглавление

1. Цель работы.....3
2. Ход выполнения работы.....3
    2.1 Часть 1. Инфраструктура и создание индекса.....3
    2.2 Часть 2. Синхронизация данных.....4
    2.3 Часть 3. Реализация полнотекстового поиска.....7
    2.4 Часть 4. Разработка UI-компонента поиска.....8
3. Вывод.....10
4. Ответы на контрольные вопросы.....11

1. Цель работы

Настроить индексацию сущности «Документ», реализовать механизм синхронизации данных между PostgreSQL и OpenSearch, а также создать интерфейс полнотекстового поиска.

2. Ход выполнения работы

2.1 Часть 1. Инфраструктура и создание индекса

Обновленное содержание файла `docker-compose.yml` представлено в листинге ниже:

```
services:
  db:
    image: postgres:16
    container_name: vrazhenko_do_postgres
    restart: always
    environment:
      POSTGRES_USER: vrazhenko_do
      POSTGRES_PASSWORD: password0000
      POSTGRES_DB: practice_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  opensearch:
    image: opensearchproject/opensearch:2.11.1
    container_name: vrazhenko_do_opensearch
    restart: always
    environment:
      - discovery.type=single-node
      - DISABLE_SECURITY_PLUGIN=true
      - OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
      - "9600:9600"
    volumes:
      - opensearch_data:/usr/share/opensearch/data

volumes:
  postgres_data:
  opensearch_data:
```

Скриншот терминала, работающих контейнеров и ответа на запрос представлен на Рисунке 1.

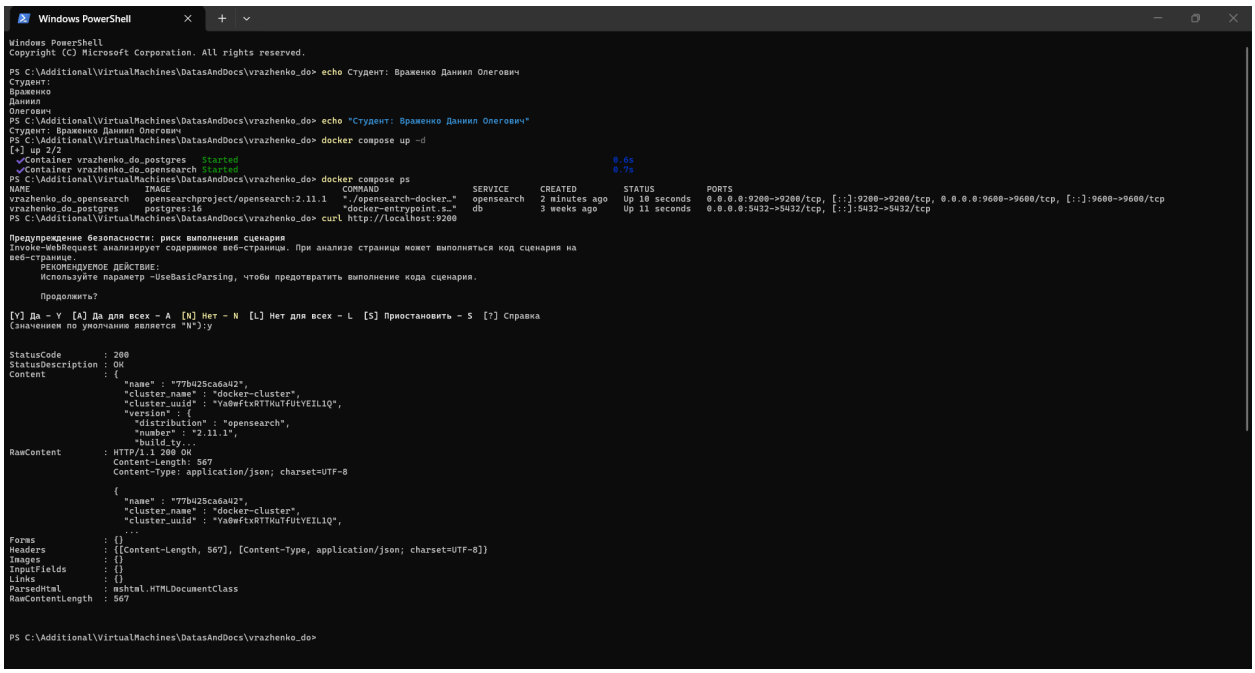


Рисунок 1 – Результаты выполненных в терминале действий

Результат создания индекса и проверки маппинга представлен на Рисунке 2.

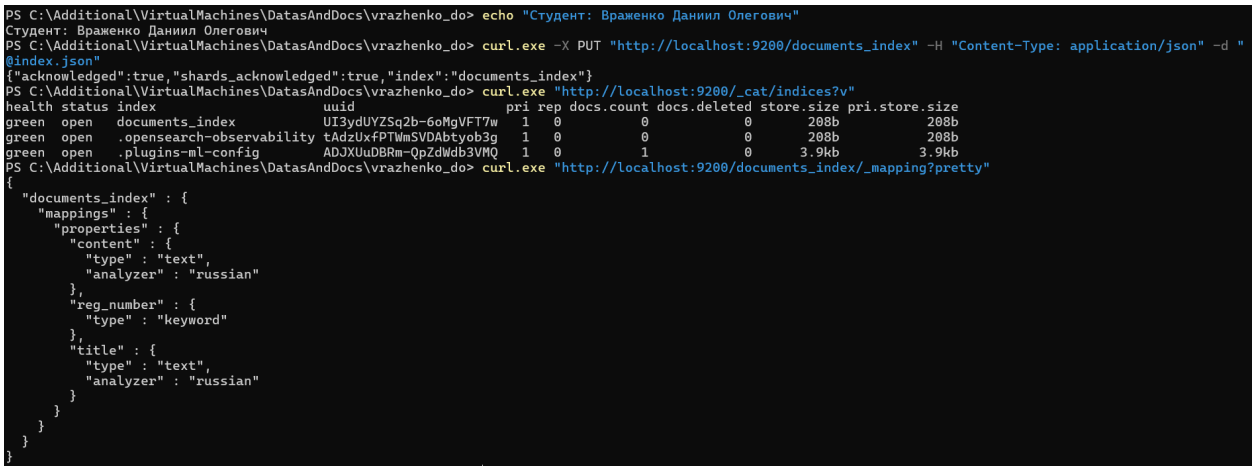


Рисунок 2 – Результаты создания индекса и проверки маппинга

2.2 Часть 2. Синхронизация данных

Добавленная зависимость в файл `pom.xml` представлена в листинге ниже:

```
        <dependency>
            <groupId>org.opensearch.client</groupId>

<artifactId>spring-data-opensearch-starter</artifactId>
            <version>3.1.0</version>
```

```
        </dependency>
```

Добавленная строка с адресом OpenSearch в файле `application.properties` представлена в листинге ниже:

```
spring.data.opensearch.uris=http://localhost:9200
```

Добавленные строки в класс `Document.java` представлены в листинге ниже:

```
    @Column(name = "content", columnDefinition = "TEXT")
    private String content;
...
    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }
```

Содержимое нового DTO-класса `DocumentIndex.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import org.springframework.data.annotation.Id;
import
org.springframework.data.elasticsearch.annotations.Document;
import
org.springframework.data.elasticsearch.annotations.Field;
import
org.springframework.data.elasticsearch.annotations.FieldType;

@Document(indexName = "documents_index")
public class DocumentIndex {

    @Id
    private Long id;

    @Field(type = FieldType.Text, analyzer = "russian")
    private String title;

    @Field(type = FieldType.Keyword)
    private String regNumber;

    @Field(type = FieldType.Text, analyzer = "russian")
    private String content;

    public DocumentIndex() {}

    public Long getId() { return id; }
```

```
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getRegNumber() { return regNumber; }
    public void setRegNumber(String regNumber) { this.regNumber
= regNumber; }
    public String getContent() { return content; }
    public void setContent(String content) { this.content =
content; }
}
```

Содержимое нового интерфейса `DocumentIndexRepository.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import
org.springframework.data.elasticsearch.repository.Elasticsearch
Repository;

public interface DocumentIndexRepository extends
ElasticsearchRepository<DocumentIndex, Long> {}
```

Обновленные строки в классе `DocumentService.java` представлены в листинге ниже:

```
    private final DocumentIndexRepository
documentIndexRepository;

    public DocumentService(DocumentRepository
documentRepository,
                           DocumentIndexRepository
documentIndexRepository) {
        this.documentRepository = documentRepository;
        this.documentIndexRepository = documentIndexRepository;
    }

    public Document saveDocument(Document document) {
        Document savedDoc = documentRepository.save(document);

        DocumentIndex indexDto = new DocumentIndex();
        indexDto.setId(savedDoc.getId());
        indexDto.setTitle(savedDoc.getTitle());
        indexDto.setRegNumber(savedDoc.getRegNumber());
        indexDto.setContent(savedDoc.getContent());

        documentIndexRepository.save(indexDto);

        return savedDoc;
    }
```

Новый столбец в классе `DocumentListView.java` представлен в листинге ниже:

```
        grid.addColumn(Document::getContent)
                .setHeader("Содержание")
                .setWidth("500px")
                .setFlexGrow(1);
```

7 добавленных документов представлены на Рисунке 3.

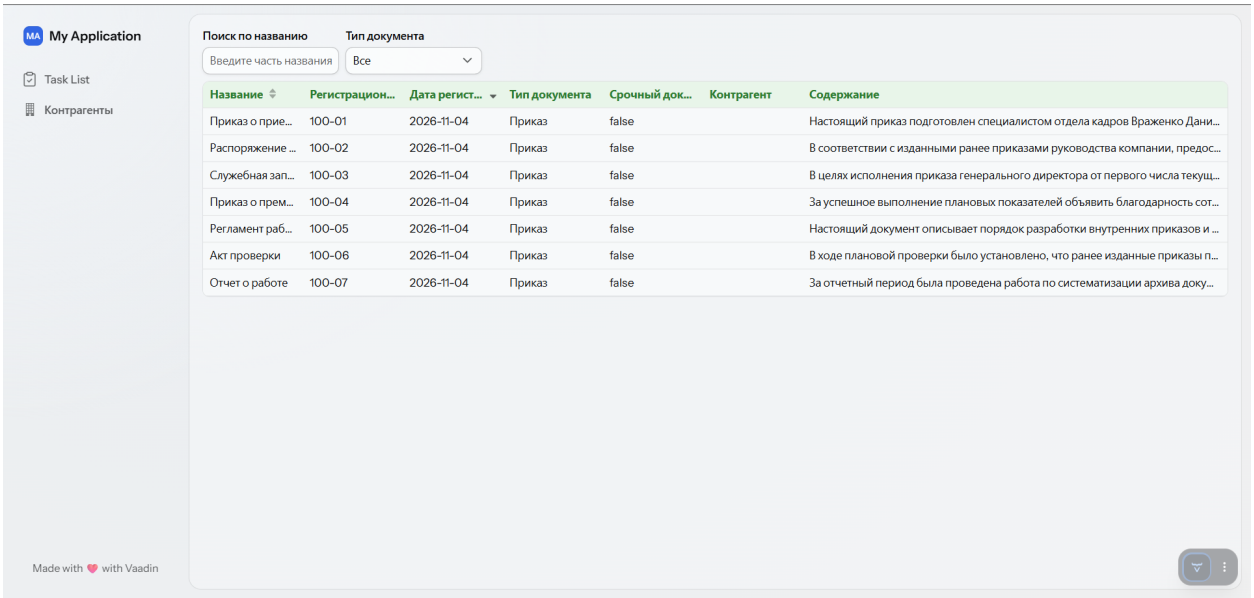


Рисунок 3 – Добавленные документы

2.3 Часть 3. Реализация полнотекстового поиска

Результат поиска через REST-запрос представлен на Рисунке 4.

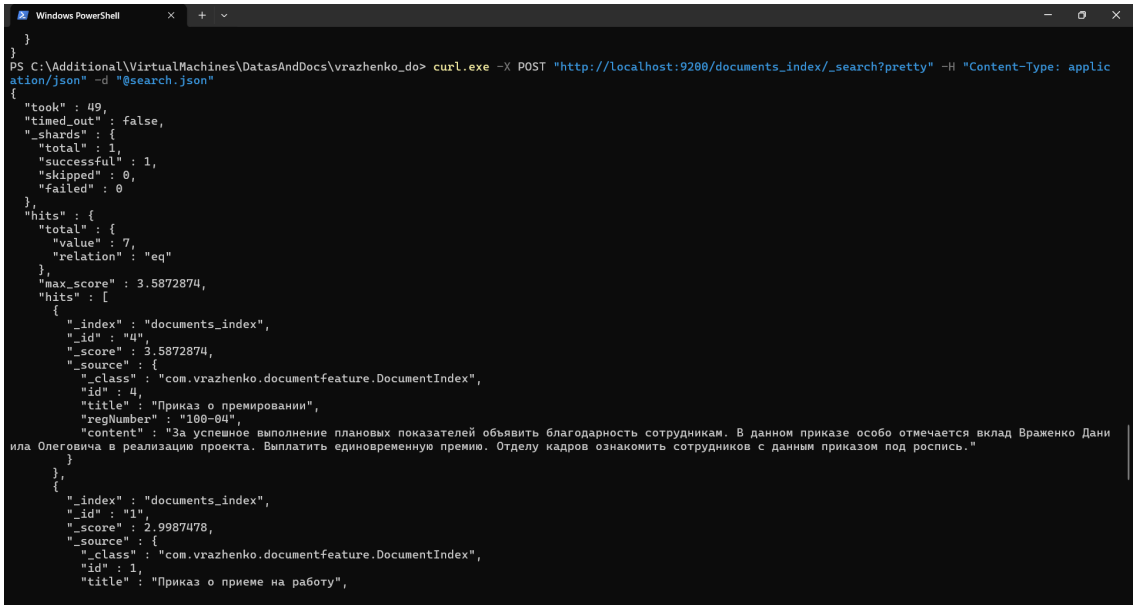


Рисунок 4 – Результат поиска через REST-запрос

Java-метод search() представлен на Рисунке 5.

```java
public List<DocumentIndex> search(String query) {  new *
    if (query == null || query.isBlank()) {
        return List.of();
    }

    var multiMatch = QueryBuilders.multiMatchQuery(query)
            .field( field: "title",  boost: 3.0f)
            .field( field: "content",  boost: 1.0f);

    var searchQuery = new NativeSearchQueryBuilder()
            .withQuery(multiMatch)
            .withMaxResults(50)
            .build();

    SearchHits<DocumentIndex> searchHits =
            elasticsearchOperations.search(searchQuery, DocumentIndex.class);

    return searchHits.getSearchHits().stream()  Stream<SearchHit<...>>
            .map(SearchHit::getContent)  Stream<DocumentIndex>
            .collect(Collectors.toList());
}
```

Рисунок 5 – Java-метод search()

2.4 Часть 4. Разработка UI-компонента поиска

Обновленное содержание класса `DocumentListView.java` представлено в листинге ниже:

```
    private final Grid<DocumentIndex> searchGrid = new
Grid<>(DocumentIndex.class);

    private final TextField searchField = new TextField("Поиск
по названию (JPA)");
...
        fullTextSearchField.setPlaceholder("Введите
запрос...");
        fullTextSearchField.setClearButtonVisible(true);

fullTextSearchField.setValueChangeMode(ValueChangeMode.LAZY);
        fullTextSearchField.setWidth("400px");
        fullTextSearchField.addValueChangeListener(e ->
updateSearchResults());
...
    private void configureSearchGrid() {
        searchGrid.removeAllColumns();

        searchGrid.addColumn(DocumentIndex::getTitle)
                .setHeader("Название")
                .setWidth("220px");

        searchGrid.addColumn(DocumentIndex::getRegNumber)
```

```
                .setHeader("Рег. номер")
                .setWidth("120px");

        searchGrid.addComponentColumn(doc -> {
                    Div wrapper = new Div();
                    String content = doc.getContent() != null ?
doc.getContent() : "";
                    String query =
fullTextSearchField.getValue();

                    if (query != null && !query.isBlank()) {
                        String highlighted =
content.replaceAll(
                                "(?i)(" + Pattern.quote(query)
+ ")",
                                "<b style='background-color:
yellow; padding: 1px 2px; border-radius: 2px;'>$1</b>"
                        );

wrapper.getElement().setProperty("innerHTML", highlighted);
                    } else {
                        wrapper.setText(content);
                    }
                    return wrapper;
                })
                .setHeader("Содержание")
                .setWidth("500px")
                .setFlexGrow(1);

        searchGrid.setSizeFull();
    }
...
    private void updateSearchResults() {
        String query = fullTextSearchField.getValue();

        if (query == null || query.isBlank()) {
            searchGrid.setVisible(false);
            mainGrid.setVisible(true);
        } else {
            mainGrid.setVisible(false);
            searchGrid.setVisible(true);

            List<DocumentIndex> results =
documentService.search(query);
            searchGrid.setItems(results);
        }
    }
```

Результат поиска слова из середины текста представлен на Рисунке 6.

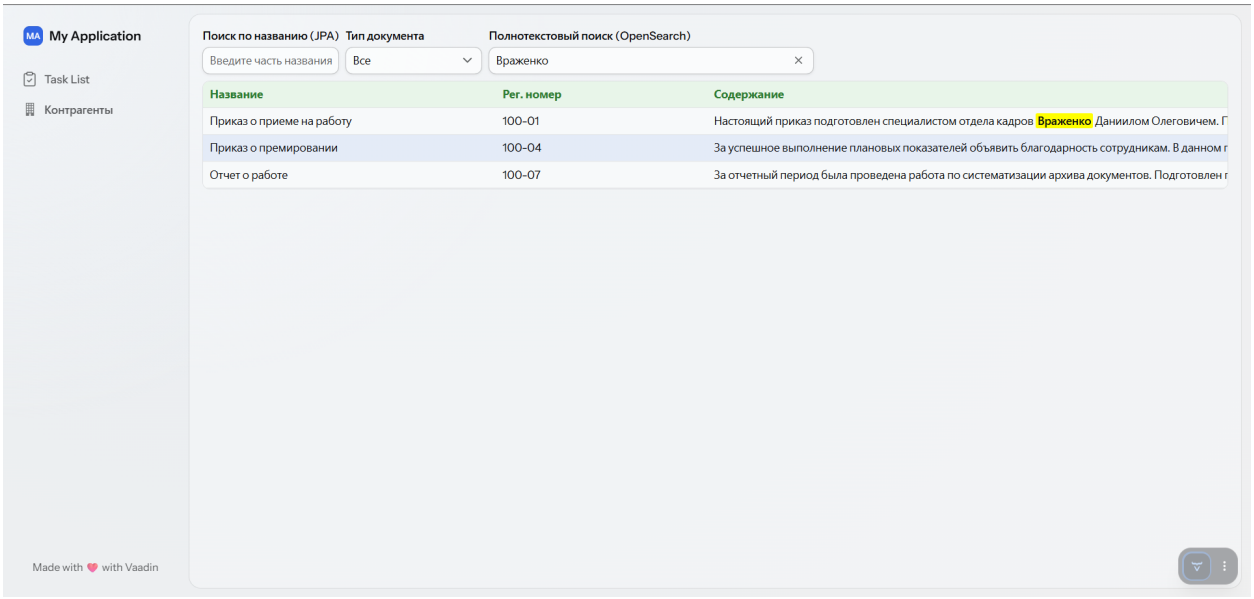


Рисунок 6 – Поиск слова из середины текста

Результат поиска по корню и демонстрации boosting представлен на Рисунке 7.

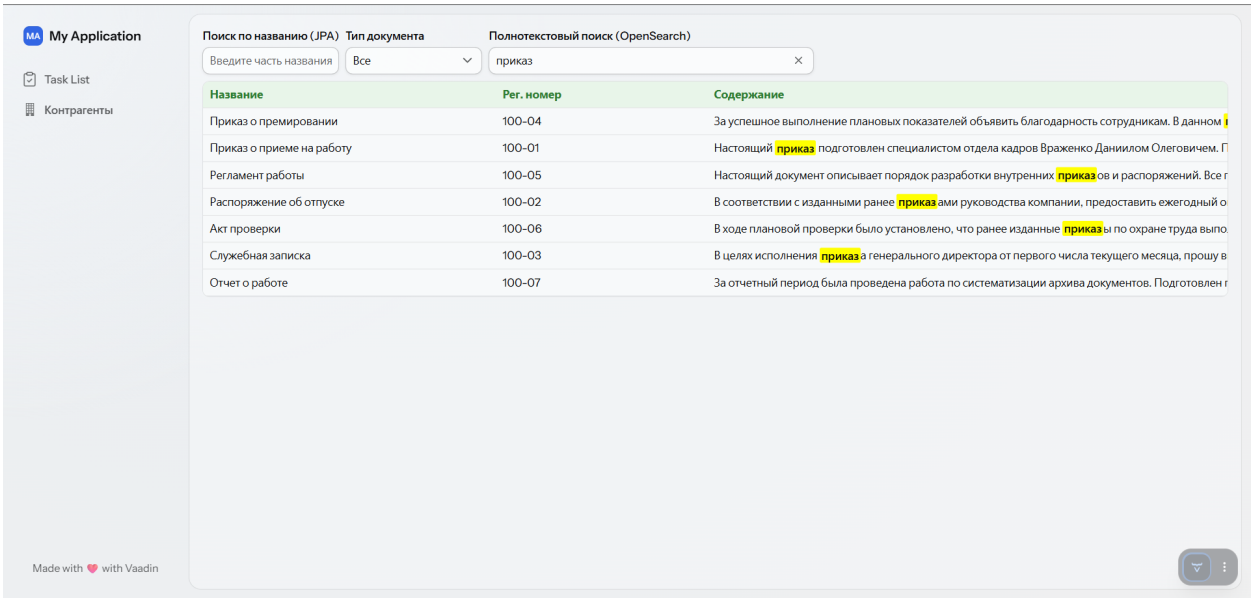


Рисунок 7 – Поиск по корню и демонстрация boosting

3. Вывод

В ходе работы настроена интеграция PostgreSQL с OpenSearch: добавлен контейнер, создан индекс `documents_index` с анализатором `russian`, реализована синхронизация при сохранении документа (индексация после `repository.save()` для получения реального ID). В Java реализован поиск

через `MultiMatchQuery` с boosting `title^3`, в Vaadin – панель поиска с подсветкой найденных фрагментов. Получены навыки работы с поисковыми системами и их интеграции с реляционной БД.

4. Ответы на контрольные вопросы

1. Почему поиск через SQL-оператор LIKE '%слово%' неэффективен для больших объемов данных?

Ведущий % не позволяет использовать B-Tree индекс – СУБД делает полное сканирование таблицы (Full Table Scan). Кроме того, LIKE не учитывает морфологию и регистр, а сравнение идёт побайтово.

2. Что такое инвертированный индекс и из каких двух основных частей он состоит?

Структура «слово – список документов, где оно встречается». Состоит из двух частей:

• Словарь (Term Dictionary) – отсортированный список уникальных слов.

• Список вхождений (Postings List) – для каждого слова: ID документов, частота (TF), позиции.

3. Чем отличаются типы полей text и keyword в маппинге OpenSearch? Для каких полей в вашей работе (title, content, reg_number) применен каждый тип и почему?

• text – строка разбивается на токены анализатором, используется для полнотекстового поиска.

• keyword – строка хранится целиком, используется для точного поиска, фильтрации, сортировки и агрегаций.

В работе: title и content – text (нужен поиск по словам с морфологией); reg_number – keyword (уникальный идентификатор, нужен точный поиск).

4. Для чего в практической работе при создании индекса указывается number_of_replicas = 0? Что произойдет со статусом кластера, если оставить 1 на одном узле?

Реплика не может разместиться на том же узле, что и Primary-шард. На одноузловом кластере при replicas = 1 индекс перейдёт в статус yellow (данные доступны, но отказоустойчивости нет). При 0 – статус green.

5. Что делает анализатор russian и какие три ключевых токен-фильтра он использует в своей цепочке?

Приводит русский текст к нормализованному виду для поиска. Три ключевых фильтра:

- Lowercase – приведение к нижнему регистру.
- Stop – удаление стоп-слов («и», «в», «на»).
- Snowball – стемминг (отсечение окончаний).

6. В чем разница между стеммингом и лемматизацией? Какой из этих подходов используется во встроенном анализаторе russian?

- Стемминг – эвристическое отсечение окончаний по правилам; быстро, но результат не всегда реальное слово.
- Лемматизация – приведение к словарной форме с помощью морфологического словаря; медленнее, но точнее.

В анализаторе russian используется стемминг (фильтр Snowball).

7. Для чего нужен MultiMatchQuery и как в нем реализуется boosting (например, синтаксис title^3)?

MultiMatchQuery ищет один запрос сразу по нескольким полям (у нас – title и content). Boosting задаётся весом поля через синтаксис поле^вес (например, title^3) или явно через .field("title", 3.0f). Документы с совпадением в заголовке получают более высокий score и оказываются выше в выдаче.

8. Почему в коде Java индексацию документа в OpenSearch нужно вызывать после repository.save(), а не до него? Что произойдет, если проиндексировать объект с null в поле ID?

repository.save() генерирует первичный ключ (@GeneratedValue) и присваивает его объекту. Если индексировать до сохранения, id будет null, и OpenSearch сгенерирует случайный UUID. При повторной индексации (например, при обновлении) в индексе появится дубликат: одна запись со случайным ID, другая – с реальным.

9. Зачем в Spring Data OpenSearch используется аннотация @Document и чем она отличается от JPA-аннотации @Entity?

• @Document (Spring Data OpenSearch) – помечает класс как сущность поискового индекса, задаёт indexName в OpenSearch.

• @Entity (JPA) – помечает класс как сущность реляционной БД, отображаемую на таблицу.

Работают в разных контекстах: NoSQL-поисковик vs реляционная СУБД.

10. Объясните разницу между контекстом Query и Filter в языке запросов OpenSearch. Что из них кэшируется?

• Query context – вычисляет score (релевантность) для сортировки. Не кэшируется.

• Filter context – только «да/нет» (прошёл/не прошёл фильтр), score не считает. Кэшируется в памяти, работает быстрее.

11. Как работает алгоритм ранжирования BM25? Какие три основных фактора он учитывает при вычислении релевантности документа?

Вероятностный алгоритм ранжирования. Учитывает:

• TF (Term Frequency) – как часто слово встречается в документе (с убывающей полезностью).

• IDF (Inverse Document Frequency) – редкость слова в индексе.

• Длина поля – нормализация (совпадение в коротком заголовке весит больше, чем в длинном тексте).

12. Почему подход Dual Writes (одновременная запись в PostgreSQL и OpenSearch из кода) считается ненадежным? Какую технологию (CDC) используют в production для синхронизации?

При Dual Writes нет распределённой транзакции: если запись в PostgreSQL прошла, а в OpenSearch – упала, данные рассогласуются. Надёжное решение – CDC (Change Data Capture): инструмент (например, Debezium) читает лог транзакций БД (WAL) и асинхронно передаёт изменения в OpenSearch.

13. Что такое шардирование и репликация в OpenSearch? Чем отличается Primary-шард от Replica-шарда?

• Шардирование – горизонтальное разделение индекса на части для распределения по узлам и масштабирования.

• Репликация – создание копий шардов для отказоустойчивости.

Primary-шард принимает запись и чтение. Replica-шард – копия, используется только для чтения (распределение нагрузки); при падении Primary повышается до Primary.

14. Как в практической работе реализована подсветка (highlighting) найденных фрагментов? В каком разделе JSON-запроса она настраивается?

В JSON-запросе – в разделе highlight (указывается поле content, теги /, размер фрагмента). В нашем приложении для наглядности сделана клиентская подсветка в Vaadin: вхождения запроса оборачиваются в <b

style='background-color: yellow;'> через replaceAll и выводятся через Div.getElement().setProperty("innerHTML", ...).

15. В каких случаях НЕ стоит использовать OpenSearch (назовите хотя бы две причины) и когда лучше ограничиться возможностями реляционной БД?

• OLTP / транзакционные данные – где нужны ACID, связи между таблицами и строгая консистентность (финансы, склад).

• Малые объёмы данных – если записей тысячи, обычная БД с индексами справится быстрее и без поддержки отдельного сервиса.

Также лучше ограничиться реляционной БД, если не требуется сложный полнотекстовый поиск с морфологией – достаточно LIKE или простых фильтров.